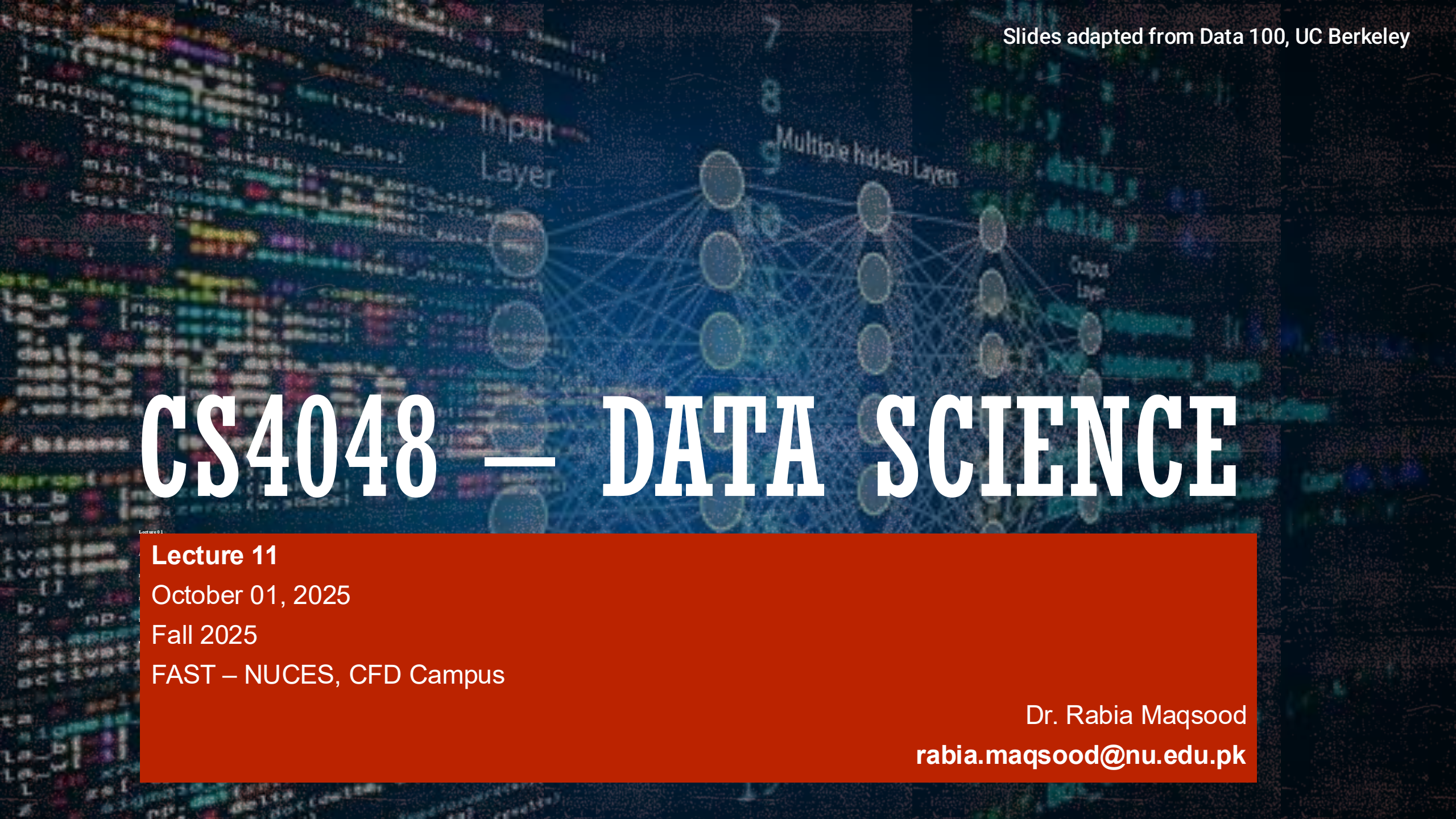


CS4048 — DATA SCIENCE



Lecture 11

October 01, 2025

Fall 2025

FAST – NUCES, CFD Campus

Dr. Rabia Maqsood

rabia.maqsood@nu.edu.pk

TODAY'S TOPICS

- Data Wrangling/Cleaning
 - Data quality issues: quick recap
 - Encoding categorical data
 - Outliers
 - Data visualization

DATA QUALITY ISSUES

- Real data is inevitable of quality issues.
- What kinds of data quality problems?
- How can we detect problems with the data?
- What can we do about these problems?
- Examples of data quality problems:
 - Inconsistent data values/wrong data
 - Duplicate data
 - Missing values
 - Noise and outliers
 - Fake data

FAITHFULNESS: DO I TRUST THIS DATA?

Fully Duplicated Records or Fields

Identify and ignore/drop.

Labeling or Spelling Errors

Apply corrections. Only ignore if you have to.

Missing data

Need to think carefully about **why** the data is missing.

Examples

" "	1970, 2000
0 , -1	NaN
999, 12345	Null

NaN: "Not a Number"

Real zero or NaN placeholder? Sometimes both!

MISSING DATA: APPROACHES

A. Keep as NaN

- A good default.
- If qualitative/categorical → Create a "Missing" category.

B. Drop records with missing values

- Typically a bad default!
- Temperature probe went offline for a minute → Likely **missing at random** → OK to drop
- Police officer never records outcomes of vehicle stops → Likely not missing at random

C. Imputation/Interpolation: Infer missing values

- **Mean/median imputation:** replace NaN with mean/median
- **Hot deck imputation:** use a random non-NaN value
- **Regression imputation:** use a model to predict value
- **Multiple imputation:** multiple random values + check sensitivity

(beyond this course)

DATA TYPES

- Numeric data: continuous vs. discrete
- Categorical data: ordinal vs. nominal
- Data/time



ENCODING CATEGORICAL DATA

ENCODING CATEGORICAL DATA

- Most Machine Learning algorithms prefer to work with numerical data, so we need to convert categories from text to numbers.
- There are multiple approaches:
 1. Label Encoding
 2. Ordinal Encoding
 3. One-Hot Encoding
 4. Binary Encoding
 5. Frequency Encoding

ENCODING TECHNIQUES (CONTD.)

- 1. Label encoding
 - Label encoding converts each category in a feature into a unique integer. This is useful for nominal categorical data where no order exists among the categories.

- Example: for a feature `Color = ["Red", "Blue", "Green", "Red", "Blue"]`

Label encoding would convert: "Red" $\rightarrow$ 0, "Blue" $\rightarrow$ 1, "Green" $\rightarrow$ 2

- This method assigns an integer to each category, but it doesn't preserve any relationships between categories.

NOTEBOOK

ENCODING TECHNIQUES (CONTD.)

- 2. Ordinal encoding
- Ordinal encoding is used when the categories have a natural order (ordinal data), such as "Low", "Medium", "High". This method assigns a numeric value to each category, respecting the inherent order.
- Example: For a feature Size = ["Small", "Medium", "Large"], the categories would be encoded as:
 - "Small" → 0
 - "Medium" → 1
 - "Large" → 2

NOTEBOOK

ENCODING TECHNIQUES (CONTD.)

- 3. One-Hot encoding

- One-Hot encoding creates a new binary column for each category in the feature. Each column will have a 1 if the category is present, and 0 otherwise. This method is best for nominal data, where categories don't have any order.

- Example:

For the feature Color = ["Red", "Blue", "Green"], One-Hot encoding would create 3 new columns, one for each color:

- "Red" $\rightarrow [1, 0, 0]$
- "Blue" $\rightarrow [0, 1, 0]$
- "Green" $\rightarrow [0, 0, 1]$

NOTEBOOK

ENCODING TECHNIQUES (CONTD.)

- 4. Binary encoding
 - Binary encoding is used for high-cardinality categorical data (data with many categories). It first converts categories into numeric values and then encodes them into binary form.
 - Example: For a feature Category = ["A", "B", "C", "D", "E"], binary encoding could result in:
 - "A" → 00001
 - "B" → 00010
 - "C" → 00011
 - "D" → 00100
 - "E" → 00101

NOTEBOOK

ENCODING TECHNIQUES (CONTD.)

- 5. Frequency encoding
 - Frequency encoding assigns a number to each category based on how frequently it appears in the dataset. It replaces each category with its frequency in the dataset.
 - Example: For the feature Fruit = ["Apple", "Banana", "Apple", "Orange", "Banana"], frequency encoding would replace:
 - "Apple" → 2 (because it appears 2 times)
 - "Banana" → 2 (because it appears 2 times)
 - "Orange" → 1 (because it appears 1 time)

NOTEBOOK

SUMMARY

1. **Label Encoding:** Assigns an integer to each category. Suitable for nominal data (e.g., red, green, blue).
 - Use: When data has no order.
2. **Ordinal Encoding:** Assigns integer values to categories based on their order. Suitable for ordinal data (e.g., Small, Medium, Large).
 - Use: When categories have a meaningful order.
3. **One-Hot Encoding:** Converts each category into a binary vector. Best for nominal data (e.g., color, product type).
 - Use: When categories do not have a natural order.
4. **Binary Encoding:** Converts categories to binary form, used for high-cardinality categorical variables.
 - Use: When the categorical variable has many levels.
5. **Frequency Encoding:** Replaces each category with the frequency of its occurrence in the dataset.
 - Use: When category frequency matters or for high-cardinality features.



21

OUTLIER DETECTION

OUTLIER

- An outlier is a data point that differs significantly from the rest of the data.
- Value(s) that lie far away from the mean or other observations in a dataset, and they can distort statistical analyses, introduce bias, and affect the performance of machine learning models.
- Understanding and handling outliers properly is crucial in data preprocessing.

TYPES OF OUTLIERS

- 1. Point Outliers

- Individual data points that are much larger or smaller than the other data points in a dataset.
- Example: In a dataset of house prices, most houses are priced between \$100,000 and \$500,000, but one house is priced at \$50 million. The \$50 million house is a point outlier.
- Point outliers can disproportionately influence measures like the mean, variance, and correlation.

TYPES OF OUTLIERS

- 2. Contextual Outliers
 - A.k.a **conditional outliers**
 - These outliers are valid data points in one context, but are considered outliers when viewed in another context. The meaning of "outlier" depends on the surrounding data or environment.
 - Example: A temperature of 100°F in summer might be perfectly normal, but in winter, the same temperature could be considered an outlier.
 - Contextual outliers are particularly relevant in time series data or seasonal data.

TYPES OF OUTLIERS

- 3. Collective Outliers

- A set of data points that together deviate significantly from the rest of the data, but individually, each data point might not seem like an outlier.
- Example: In a time series dataset, a group of consecutive data points that suddenly spike or drop significantly could be a collective outlier, even if individual points aren't too extreme.
- Collective outliers are important in time series analysis or when analyzing grouped data.

DETECTING OUTLIERS

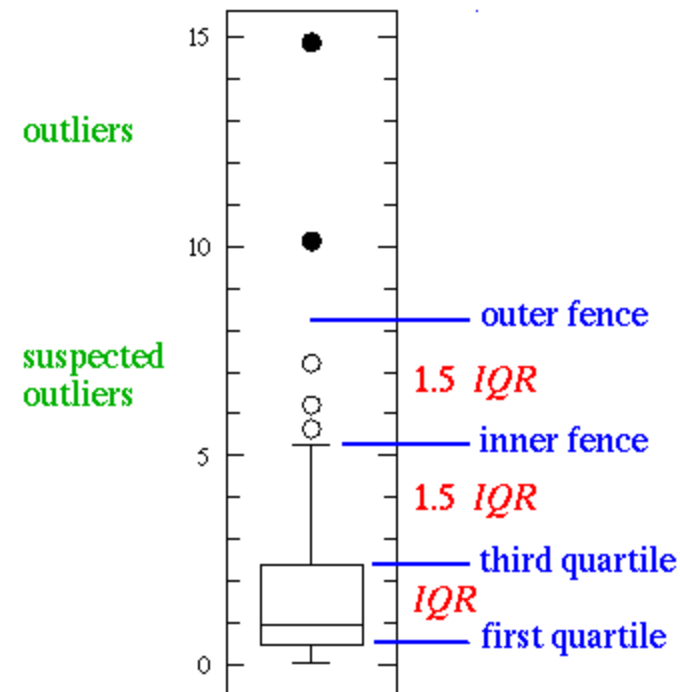
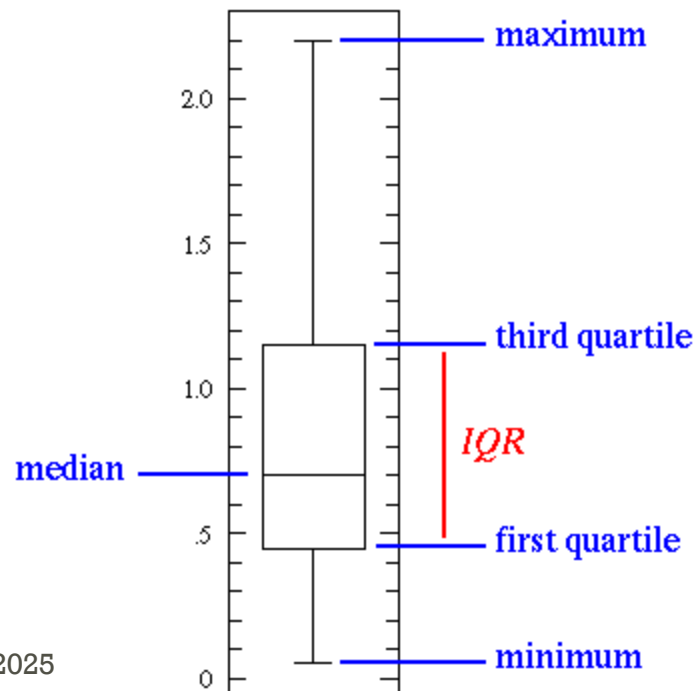
- Point outliers can be detected using methods like Z-scores or **IQR (Interquartile Range)**.
- Contextual outliers are harder to detect with simple statistical methods.
 - Typically identified in datasets with temporal or seasonal patterns, such as time series data.
- Collective outliers can be detected in time series data by looking for sudden spikes or drops in multiple consecutive points.

WHY OUTLIER MATTER

- Outliers can:
 - **Skew statistical measures:** The mean, standard deviation, and correlation can be heavily influenced by outliers.
 - **Affect machine learning models:** Some algorithms (like linear regression, k-NN, or distance-based models) are sensitive to outliers, which can lead to poor model performance.
 - **Distort visualizations:** Outliers can make it difficult to see the pattern in data when plotting graphs like histograms, scatter plots, or box plots.

BOXPLOT

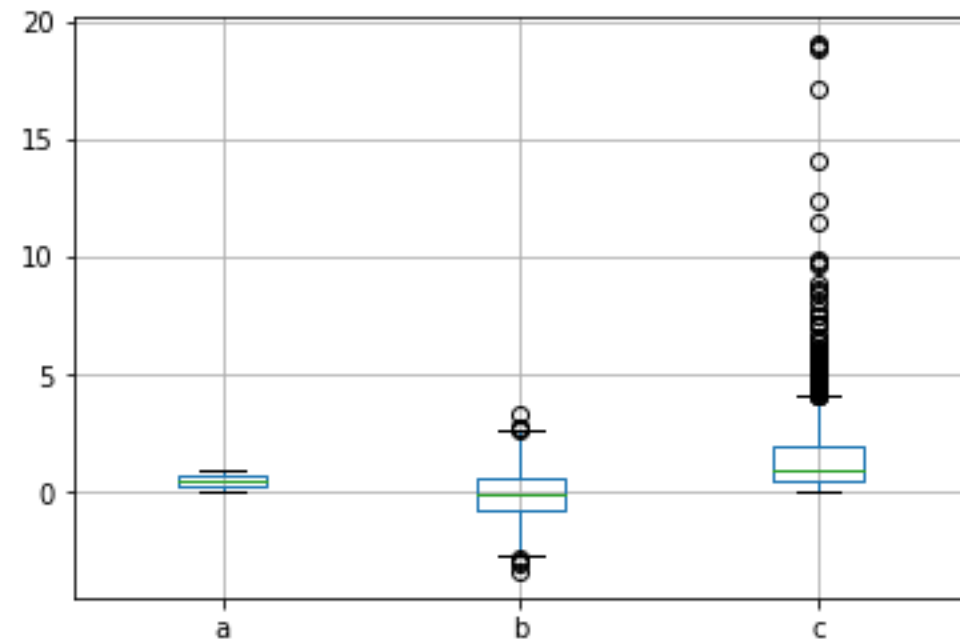
- The box plot (a.k.a. box and whisker diagram) is a standardized way of displaying the distribution of data based on the five number summary: minimum, first quartile, median, third quartile, and maximum. In the simplest box plot the central rectangle spans the first quartile to the third quartile (the *interquartile range* or *IQR*). A segment inside the rectangle shows the median and "whiskers" above and below the box show the locations of the minimum and maximum.



BOXPLOT EXAMPLE

```
df=DataFrame({'a': np.random.rand(1000),  
              'b': np.random.randn(1000),'c': np.random.lognormal(size=(1000))})  
print(df.head())  
df.boxplot()
```

	a	b	c
0	0.316825	-1.418293	2.090594
1	0.451174	0.901202	0.735789
2	0.208511	-0.710432	1.409085
3	0.254617	-0.637264	2.398320
4	0.256281	-0.564593	1.821763



HOW TO HANDLE OUTLIERS

- **Remove Outliers:** If the outliers are erroneous or irrelevant, you can remove them from the dataset.

Python Code:

```
# Removing outliers from the dataset (Z-score method)

df_no_outliers = df[(z_scores < 3)]
```

- **Capping or Clipping:** You can limit the outliers by setting them to a maximum or minimum value.

Python Code:

```
# Capping outliers at a specified upper and lower bound
df['Value'] = np.clip(df['Value'], df['Value'].quantile(0.05),
                      df['Value'].quantile(0.95))
```

HOW TO HANDLE OUTLIERS

- **Transformation:** Applying transformations like log, square root, or Box-Cox can help reduce the impact of outliers, especially in skewed data.

Python Code:

```
# Apply log transformation
df['Transformed'] = np.log(df['Value'] + 1)

# Log transformation (adding 1 to avoid log(0))
```

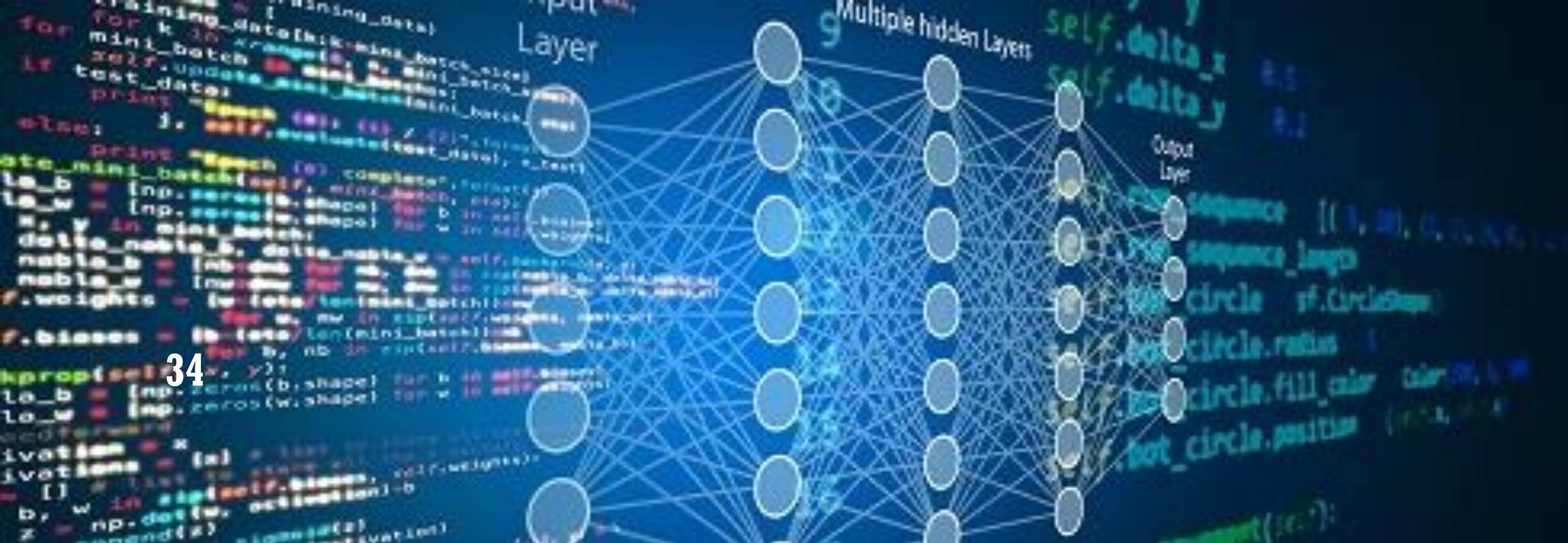
- **Imputation:** For some types of data, you may decide to impute outliers with meaningful values, such as the mean, median, or other model-based imputations.

Python Code:

```
# Impute outliers with the median value

df['Value'] = df['Value'].apply(lambda x: np.median(df['Value'])
```

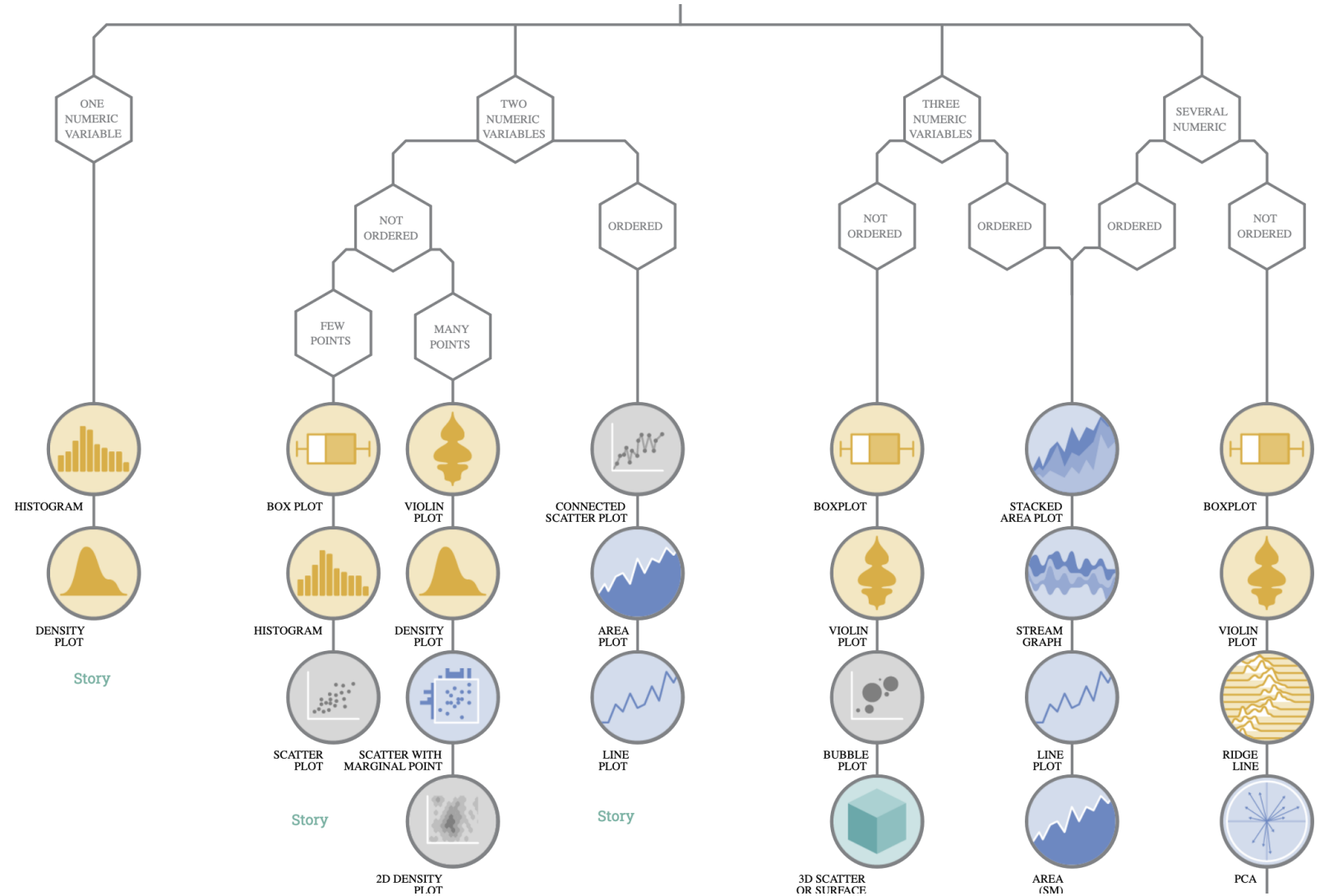
```
CS4048-Fall 2025> upper_bound else x)
```



DATA VISUALIZATION

DATA VISUALIZATION

Explore **Data to Viz**
(<https://www.data-to-viz.com>)



READING MATERIAL

- Wrangling Dataframes: https://learningds.org/ch/09/wrangling_intro.html
- Chapter#2: Hands-On Machine Learning, Aurélien Géron